

aggregate.py

```
1  # Aggregate.py
2  # This script handles the aggregation of utilization constraints for optimization
3  # -----
4  # INPUT  <- Utilization Constraints as list (c_util_agg)
5  #        <- Method: is chosen in Main
6  #        <- P-value: is chosen in Main
7  # OUTPUT -> Aggregated Utilization Constraints
8  # -----
9  import numpy as np
10
11 # Define the class
12 class ConstraintAggregate:
13     def __init__(self, method=None, p_value = None, rho_value = None, relaxation = None):
14         # Read Method
15         self.method = method
16         self.rho = rho_value
17
18         # Read P value
19         self.p = p_value
20
21     # Function for handling aggregation
22     def agg_output(self, g, c_scale = 1.0):
23         # Relaxation parameter for P-norm methods (0<eps<1)
24         relaxation = 0
25         g = np.asarray(g, dtype = float)
26         g_k = np.maximum(-(g - relaxation), 0.0)
27         g_max = np.max(g_k)
28
29         # No Aggregate Method
30         if self.method is None:
31             return g, None, None
32
33         # P-norm method
34         if self.method == "P-norm":
35             p = float(self.p)
36             v = (np.sum(g_k**p))**(1/p)
37             print(f"Max Const: {g_max:.3f} | P-norm: {v:.3f}")
38
39         # P-norm mean method
40         if self.method == "P-norm-mean":
41             p = float(self.p)
42             n = np.size(g)
43             g_k = np.maximum(-(g - relaxation), 0.0)
44             v = (1/n*np.sum(g_k**p))**(1/p)
45
46             print(f"Max Const: {g_max:.3f} | P-norm-mean: {v:.3f}")
47
48
49
50
51     v_scaled = c_scale * v
52
53     return -v_scaled, v, g_max
54
55
```